

Experiment 4

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes. Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Exp.1

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    char command[100];
```

```
    printf("Enter a Linux command: ");
```

```
    scanf("%s", command);
```

```
    pid_t pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
    return 1;
}
else if (pid == 0)
{
    printf("Child Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    execlp(command, command, NULL);

    perror("exec failed");
    exit(1);
}
else
{
    printf("Parent Process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
```

```

}
ashwithreddy_earla@AshwithReddy:~$ nano exper4.c
ashwithreddy_earla@AshwithReddy:~$ gcc exper4.c -o exper4
ashwithreddy_earla@AshwithReddy:~$ ./exper4
Enter a Linux command: ls
Parent Process
Parent PID: 552
Child PID: 555
Child Process
Child PID: 555
Parent PID: 552
Experiment2.c      exec          expl.program7    experiment4.c    input.txt       sample.c         ssize_t
Experiment5        exec.c        expl.program8    experiment5.c    mkfifo         sample.c.save   text.c
Experiment5.c      expl         expl.program8.c  experiment6.c    myfifo        sample.txt      text.c.save
Experiment6server.c expl.c        exper4           experiment7.c    open          skill           write
HELLO             expl.program1.c exper4.c         fork            open.c        skill.c         write.c
OSSP              expl.program2.c experiment       fork.c          operating      skill3
OSSP.c            expl.program3.c experiment2     fork_exec       output.txt     skill3.c
close             expl.program4.c experiment2.c   fork_exec.c     producure      skill3.c:Zone.Identifier
close.c           expl.program5.c experiment3     gcc             producure.c    skill4.c
copy.txt          expl.program5.c experiment3.c   input           read           skill4.c:Zone.Identifier
dirname           expl.program6.c experiment4     input.c         read.c         source.txt
Child process completed.

```

Exp 2.c

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution. Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    printf("Before fork():\n");
```

```
    printf("Process PID : %d\n", getpid());
```

```
    printf("Parent PPID : %d\n", getppid());
```

```
pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}
else if (pid == 0)
{
    // Child process
    printf("\n--- CHILD PROCESS ---\n");
    printf("Child PID : %d\n", getpid());
    printf("Child PPID : %d\n", getppid());
    printf("Child State : Running\n");

    sleep(5);

    printf("Child State : Terminating\n");
    exit(0);
}
else
{
    // Parent process
    printf("\n--- PARENT PROCESS ---\n");
    printf("Parent PID : %d\n", getpid());
```

```
printf("Parent PPID : %d\n", getppid());  
printf("Child PID : %d\n", pid);  
printf("Parent State : Running\n");  
  
printf("\nParent waiting for child...\n");  
  
wait(NULL);  
  
printf("Child terminated.\n");  
printf("Parent State : Running\n");  
printf("Parent State : Terminated after completion\n");  
}  
  
return 0;
```

}

```
ashwithreddy_earla@AshwithReddy:~$ nano exper4.2.c
ashwithreddy_earla@AshwithReddy:~$ gcc exper4.2.c -o exper4.2
ashwithreddy_earla@AshwithReddy:~$ ./exper4.2
Before fork():
Process PID : 777
Parent PPID : 310

--- PARENT PROCESS ---
Parent PID : 777
Parent PPID : 310
Child PID : 778
Parent State : Running

Parent waiting for child...

--- CHILD PROCESS ---
Child PID : 778
Child PPID : 777
Child State : Running
Child State : Terminating
Child terminated.
Parent State : Running
Parent State : Terminated after completion
```